



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	12 / Tipe Data Tuples

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Tuple Immutable

- Nilai yang disimpan dalam *tuple* bisa berupa apapun dan diberi indeks bilangan bulat (integer).
- *Tuple* bersifat **immutable** yang artinya anggota dalam *tuple* tidak bisa dirubah.
- *Tuple* bisa dibandingkan (*compare*) dan bersifat *hashable* sehingga bisa dimasukkan ke dalam *list* sebagai *key* pada *dictionary python*.
- Disebut *hashable* jika memiliki nilai *hash* yang tidak pernah berubah selama hidupnya (method `__hash__()`) dan bisa dibandingkan dengan benda lain (method `__eq__()` atau `__cm__()`). Obyek *Hashable* akan membandingkan dengan yang memiliki *nilai hash* sama.
- *Hashability* dapat membuat obyek yang digunakan sebagai kamus kunci dan mengatur anggotanya karena struktur data ini menggunakan nilai *hash* secara internal.
- Objek pada python *built-in* biasanya *hashable*, sementara tidak bisa berubah wadah (*list / dictionary*) tidak *hashable*.
- Objek yang merupakan *instance* dari kelas yang didefinisikan pengguna yang *hashable* secara default, mereka semua membandingkan yang tidak sama, dan nilai hash mereka adalah `id()`.

```
laprak13.py > ...
1  t1 = 'a','b','c'
2  t2 = ('a','b','c')
3  t3 = ('a',)
4
5  print(type(t1),type(t2),type(t3))
```

Gambar 13.1 : Sintaks penulisan *tuple*

- Sintaks penulisan *tuple* terdapat beberapa cara. Jika hanya 1 elemen, maka tambahkan koma (,) diakhirnya. Jika tidak ada koma (,) maka akan dianggap sebagai *string*.

```
laprak13.py > ...
1  t = tuple()
2  print(t)
```

Gambar 13.2 : Fungsi **built-in** *tuple*.

- Jika tidak diisi argument apapun, maka secara langsung akan membentuk *tuple* kosong.

```

laprak13.py > ...
1  t = tuple('angelarevaline')
2  print(t)

```

Output :

```
('a', 'n', 'g', 'e', 'l', 'a', 'r', 'e', 'v', 'a', 'l', 'i', 'n', 'e')
```

- Jika argumentnya merupakan urutan (*string*, *list*, atau *tuple*) maka outputnya akan mengembalikan nilai *tuple* dengan elemen-elemen yang berurutan.
- Kita dapat memanggil *tuple* menggunakan indeks dengan [].

```

laprak13.py > ...
1  t = tuple('angelarevaline')
2  print(t[2])

```

Output :

```
g
```

```

1  t = tuple('angelarevaline')
2  print(t[10:13])

```

Output :

```
('l', 'i', 'n')
```

```

1  t = tuple('angelarevaline')
2  print(t[0] == 'A')

```

Output :

```
False
```

```

1  t = tuple('angelarevaline')
2  t = t[0:] + ('s',)
3  print(t)

```

Output :

```
('a', 'n', 'g', 'e', 'l', 'a', 'r', 'e', 'v', 'a', 'l', 'i', 'n', 'e', 's')
```

2. Membandingkan *Tuple*

- Operator perbandingan (*compare*) bisa bekerja pada *tuple* dan model sekuensial lainnya (*list*, *dictionary*, *set*).
- Cara kerja = membandingkan elemen pertama dari setiap sekuensial yang ada.
- Apabila ditemukan kesamaan, maka akan lanjut ke elemen berikutnya. Proses ini berlangsung terus menerus hingga ditemukan suatu perbedaan.
- Fungsi ***sort*** pada python bekerja dengan cara yang sama, dimana pertama akan melakukan pengurutan berdasarkan elemen pertama. Jika ada elemen yang sama, maka akan dilanjutkan ke elemen kedua dan seterusnya. Fitur ini disebut DSU (***Decorate, Sort, Undercorate***).
- 1) ***Decorate*** = urutan (sekuensial) membangun daftar *tuple* dengan satu atau lebih key pengurutan sebelum elemen dari urutan (sekuensial).
- 2) ***Sort*** = *list tuple* menggunakan *sort* (fungsi bawaan di python).
- 3) ***Undercorate*** = melakukan ekstraksi pada elemen yang telah diurutkan pada satu sekuensial.

```

1 kalimat = 'but soft what light in yonder window breaks'
2 daftar_kata = kalimat.split()
3 t = list()
4 for kata in daftar_kata:
5     t.append((len(kata), kata))
6
7 t.sort(reverse=True)
8
9 urutan = list()
10 for length, kata in t:
11     urutan.append(kata)
12
13 print(urutan)

```

Gambar : Contoh program yang akan melakukan pengurutan kata dalam kalimat dari yang paling panjang ke paling pendek.

- Looping pertama akan membuat daftar *tuple*, yang berisi daftar kata sesuai dengan panjangnya. Fungsi **sort** akan membandingkan elemen pertama dengan list dari panjang kata yang ada, lalu lanjut ke elemen ke dua jika kondisi sesuai.
- *reverse=True* berfungsi untuk melakukan urutan secara terbalik (*descending*).
- Looping kedua akan membuat daftar *tuple* dalam sesuai urutan alfabet yang diurutkan berdasarkan panjangnya. Sehingga kata “*what*” akan muncul sebelum “*soft*” dalam list.

Output :

```
['yonder', 'window', 'breaks', 'light', 'what', 'soft', 'but', 'in']
```

3. Penugasan *Tuple*

- Salah satu fitur unik yang dimiliki Python adalah kemampuannya untuk memiliki *tuple* di sisi kiri dari statement penugasan. Hal ini memungkinkan kita untuk menetapkan lebih dari satu variabel pada sisi sebelah kiri secara berurutan.

```

1 a = ['angel', 'valine']
2 x, y = a
3
4 print(x)    #output: 'angel'
5 print(y)    #output: 'valine'

```

- Hal yang perlu diperhatikan adalah jumlah variabel di sisi kiri dan kanan harus sama

```

1 try:
2     a, b = 1, 2, 3
3 except ValueError as e:
4     print(e)    #output : too many values to unpack (expected , got 3)

```

- Contoh untuk memisahkan alamat email menjadi username dan domain.

```
1 email = 'angela.revaline@ti.ukdw.ac.id'
2 username, domain = email.split('@')
3
4 print(username) #output: angela.revaline
5 print(domain)   #output: ti.ukdw.ac.id
```

Nilai yang dikembalikan dari split terdiri dari 2 elemen yang telah dipisahkan oleh tanda '@'.

4. Dictionaries and Tuple

- Dictionaries memiliki metode **items** yang berfungsi untuk mengembalikan nilai *list* dari *tuple* dimana setiap *tuple*-nya merupakan *key-value pair* (pasangan kunci dan nilai).

```
1 dict = {'a': 23, 'j': 25, 'r': 18}
2 t = list(dict.items())
3 print(t)
```

Output :

```
[('a', 23), ('j', 25), ('r', 18)]
```

- Biasanya item yang dikembalikan nilainya tidak berurutan, sehingga kita bisa gunakan fungsi **sort()**.

```
5 t.sort()
6 print(t)
```

5. Multipenugasan dengan *dictionaries*

- Kombinasi antara **items**, **tuple assignment** dan **for** akan menghasilkan kode tertentu pada *keys* dan *values* dari *dictionary* dalam satu loop.

```
1 dict = {'a': 23, 'j': 25, 'r': 18}
2
3 for key, val in list(dict.items()):
4     print(val, key)
```

Output :

```
23 a
25 j
18 r
```

- Pada looping di atas, terdapat 2 iterasi variabel karena *item* mengembalikan *list* dari *tuple* dan **key, val** merupakan penugasan *tuple* yang melakukan iterasi berulang melalui pasangan *key-value* pada *dictionary*.
- *key* dan *value* akan bergerak maju menuju pasangan *key-value* berikutnya pada *dictionary*.

```

1 dict = {'a': 23, 'j': 25, 'r': 18}
2 l = list()
3
4 for key, val in dict.items():
5     l.append((val, key))
6 print(l)
7
8 l.sort(reverse=True)
9 print(l)

```

6. Kata yang sering muncul

- Contoh program yang menggunakan *list* dan *tuple* untuk menampilkan 10 kata yang paling sering muncul.

```

1 import string
2 handle = open('romeo-full.txt')
3 counts = dict()
4
5 for line in handle:
6     line = line.translate(str.maketrans('', '', string.punctuation))
7     line = line.lower()
8     words = line.split()
9
10    for word in words:
11        if word not in counts:
12            counts[word] = 1
13        else:
14            counts[word] += 1
15
16    first = list()
17    for key, val in list(counts.items()):
18        first.append((val, key))
19
20    first.sort(reverse=True)
21
22    for key, val in first[:10]:
23        print(key, val)

```

- Bagian awal dari program digunakan untuk membaca file dan melakukan komputasi terhadap *dictionary* yang memetakan tiap kata ke sejumlah kata dalam dokumen yang tidak berubah. Dengan menggunakan *print out counts*, list tuple (*val, key*) dibuat dan diurutkan dalam list dengan urutan terbalik.
- Nilai pertama akan digunakan sebagai perbandingan. Jika ada lebih dari satu *tuple* dengan nilai yang sama lalu akan lanjut melihat ke elemen kedua (*key*), sehingga *tuple* akan mengurutkan berdasarkan urutan abjad sesuai *key*.
- Pada bagian akhir, looping akan melakukan iterasi penugasan ganda dan mencetak 10 kata yang paling sering muncul dengan melakukan perulangan pada list **first[:10]**.

Output :

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

7. *Tuple* sebagai kunci *dictionaries*

- Tuple = hashable, sedangkan list = tidak hashable.
- Jika ingin membuat *composite key* yang digunakan dalam *dictionary*, bisa menggunakan tuple sebagai *key*.

```
1 directory = dict()
2 directory ['Revaline', 'Angela'] = 71251188
3
4 for last, first in directory:
5     print(first, last, directory[last, first])
```

Output :

```
Angela Revaline 71251188
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

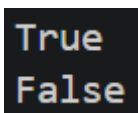
Source Code :

```
def persamaan(tuple):  
    if len(tuple) == 0:  
        return False  
    for i in range(1, len(tuple)):  
        if tuple[i] != tuple[0]:  
            return False  
    return True  
  
print(persamaan((90,90,90,90)))  
print(persamaan((90,88,90,90)))
```

Penjelasan :

1. Buat fungsi bernama **persamaan** dengan parameter bernama **tuple**.
2. Cek jika panjang tuple sama dengan 0, maka kembalikan nilai **False**.
3. Menggunakan perulangan **for** dalam range pertama hingga sepanjang tuple, cek apakah tuple indeks 0 sama dengan selanjutnya.
4. Jika tidak sama maka kembalikan nilai False, tetapi jika sama semua maka kembalikan nilai True.

Output :



```
True  
False
```


SOAL 2

Source Code :

```
def identitas_diri(data):  
    nama, nim, alamat = data  
  
    print("Data: ", data)  
    print()  
    print("NIM      :   ", nim)  
    print("NAMA      :   ", nama)  
    print("ALAMAT    :   ", alamat)  
    print()  
    print("NIM: ", tuple(nim))  
    print()  
    print("NAMA DEPAN:  ", tuple(nama.split()[0]))  
    print()  
    print("NAMA TERBALIK: ", tuple(nama.split()[::-1]))  
  
identitas_diri(('Angela   Revaline   Setiawan',   '71251188',  
                'Muntilan, Magelang'))
```

Penjelasan :

1. Buat fungsi bernama **identitas_diri** dengan parameter bernama **data**.
2. Nama, nim, dan alamat merupakan data yang akan diproses oleh program.
3. Gunakan **print** untuk mengeluarkan output sesuai yang diinginkan.
4. Gunakan fungsi **split()** untuk memisahkan setiap huruf.

Output :

```
Data: ('Angela Revaline Setiawan', '71251188', 'Muntilan, Magelang')

NIM      : 71251188
NAMA     : Angela Revaline Setiawan
ALAMAT   : untilan, Magelang

NIM: ('7', '1', '2', '5', '1', '1', '8', '8')

NAMA DEPAN: ('A', 'n', 'g', 'e', 'l', 'a')

NAMA TERBALIK: ('Setiawan', 'Revaline', 'Angela')
```

SOAL 3

Source Code :

```
def jam_email(namaFile):
    try:
        data = open(namaFile)
    except:
        return "File tidak ditemukan."
    hitungJam = {}
    for baris in data:
        if baris.startswith('From '):
            isi = baris.split()
            if len(isi) > 5:
                waktu = isi[5]
                jam = waktu.split(':')[0]
                hitungJam[jam] = hitungJam.get(jam, 0) + 1
    hasil = sorted(hitungJam.items())
    return hasil

for jam, jumlah in jam_email("mbox-short.txt"):
    print(jam, jumlah)
```

Penjelasan :

1. Buat fungsi bernama **jam_email** dengan parameter bernama **namaFile**.
2. Gunakan try-except untuk menangani error yang terjadi. Jika error maka program akan langsung mengeluarkan output "File tidak ditemukan."
3. Buat variabel bernama **hitungJam** yang akan menyimpan data berupa **dictionary**.
4. Menggunakan fungsi **perulangan for** kita akan mengecek jika baris diawali dengan kata "From" dengan menggunakan fungsi **.startswith()**.
5. Buat variabel bernama **isi** yang akan menyimpan baris yang telah dipisahkan berdasarkan spasinya.
6. Jika baris memiliki lebih dari 5 elemen, maka waktu berupa isi elemen ke 5 dan memisahkan jamnya menggunakan **split()**.
7. Menggunakan fungsi **get** untuk menghitung jam dan menambahkannya ke dalam dictionary **hitungJam**.
8. Setelah itu urutkan menggunakan fungsi **sorted()**.
9. Gunakan fungsi **return** untuk mengembalikan nilai.
- 10.

Output :

```
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```